

Learning to Apply, Not to Memorize: Policy-in-Context Fine-Tuning Generalizes to Unseen Organizational Policies

Draft v0.6 (2026-09-01) — target: EMNLP/ACL Industry track. All numbers are measured and reproducible from this repository (pointers in §7). All arXiv IDs and titles were verified against the abstract pages; the TriMPI and PRT positioning claims were additionally confirmed against the paper bodies (Policy Override §5.2; per-policy instance splits).

Abstract

In organizations, correct behavior is defined by documents that change on the organization's schedule; an AI system that makes decisions *their* way must follow the written policy, cite the rule that applied, and defer exactly where the policy grants discretion. The dominant fine-tuning recipe internalizes a specific policy into model weights, which breaks the moment the policy changes. We study the opposite framing: supply the policy *in context* at inference time, and fine-tune a small model on the **skill of applying whatever policy it is given**. On a corpus of 34 organizational decision cases (3,400 labeled decisions generated by a deterministic rules engine over declarative ontologies), a QLoRA-tuned 7B model reaches **72.9% exact-match** (verdict, routing, parameters, and a *correct* cited rule) on policies never seen in training — evaluated leave-case-out so every policy is unseen exactly once — versus 15.0% for the untrained model under identical serving (cluster-bootstrap 95% CI [64.7, 80.9] vs [3.3, 26.7]). Three controls pin down what was learned: on **counterfactual policies** (approve/reject flipped, evaluated only where the flip changes the answer, so prior knowledge scores zero) the tuned model scores **88.0%**; a 3-shot in-context-learning baseline reaches only 50.0%; and training-seed variance is ± 4.8 pp. A negative control bounds the recipe: the *same* recipe on an exact arithmetic derivation task scores **0/6** — **even with chain-of-thought targets, and even for a frontier model prompted identically without tools** (both fail almost solely on high-precision division fields), closing the "train harder" objection and confirming that derivations belong outside inference — not merely behind a per-run tool call, but compiled to code so that repeat runs carry no LLM in the loop at all. Because the pipeline compiles from a work session the organization already paid for, the frontier-token break-even is the **second run**. Together these results give practitioners a concrete placement rule for small models in deterministic agent pipelines.

1. Introduction

In most organizations, correct behavior is defined by *documents* — decision policies, support runbooks, escalation matrices, grading rubrics, editorial style guides, compliance checklists — and those documents change on the organization's schedule, not the model's. Any AI system built on them therefore needs a particular decomposition: the **content** (what the document says today) must live *in context*, where it can be swapped the day it changes, while the **skill** (how to read such a document and apply it faithfully to a case) can live *in weights*. This paper measures whether that skill is actually learnable by supervised fine-tuning of a small model — and, critically, whether it transfers to documents the model has never seen.

In practice the decomposition plays out like this. The people who own these tasks are not programmers or model experts. At first they simply do their job — quotes, refund approvals, incident triage — in conversation with an agent. Those working sessions already contain the job's policies, its calculations, and its way of judging. Expressed in a declarative work language (OpenWorkLang) and run through its compiler (OpenWorkCompiler), the policies become an ontology with rules, the calculations become programs, and the judgment goes to a trained small model — yielding a task-specific execution-and-decision tool that runs on a local GPU, without a frontier LLM, at a fraction of the tokens. This paper puts the most doubtful link of that chain to the test — is judgment

actually learnable by a small model? — and backs the other links with controls and measurements (calculation belongs to programs, §5.3; the token economics, §5.4).

We study the cleanest instance of this class that still admits a deterministic grader: organizational decision policies. When an employee asks "can I give this customer a 12% discount?", the organization does not want an oracle's opinion. It wants *its own policy applied*: the ordered rules checked top-down, the first matching rule cited, the case routed to the approver the policy names, and the band the policy deliberately left open ("5–10%: AI may recommend, team lead approves") — and only that band — filled by a recommendation. Three properties follow that shape the ML problem, and each instantiates the general decomposition:

1. **The documents change faster than models.** A model that memorized policy P is wrong the day P becomes P'. The content must live *outside* the weights.
2. **Judgments must be auditable.** "Approve" is not an answer; "approve, route to director, citing `strategic_retention_band` whose condition holds on this record" is.
3. **Discretion is declared, not inferred.** The document says where the model may recommend; confidence thresholds do not.

Prior work internalizes policies into weights (Multimodal Policy Internalization / TriMPI, arXiv:2510.09474), benchmarks agents given domain policy guidelines (τ -bench, 2406.12045; RuleArena, 2412.08972), or measures robustness to rule updates (GuideBench, 2505.11368). What is missing is the combination that deployment actually needs: **fine-tune a small, locally-servable model on the *transferable skill* of applying an arbitrary in-context policy, and measure transfer to policies never seen in training.**

Declarative decision policies make the skill (ordered evaluation, first-match, grounded citation) non-trivial while keeping correctness checkable to the byte. Whether the recipe transfers to messier members of the document class — runbooks, rubrics — is an open question we scope explicitly, with a roadmap, in §6.

We contribute:

- **Task & corpus** (§3): 34 decision cases across 10 organizational functions, each declared as an ontology plus ordered rules with explicit discretion bands, compiled by a deterministic engine into 3,400 labeled decisions with cited rules and grounded conditions.
- **A strict, deterministic grader** (§3.3): exact match on verdict, route, parameters, *and* cited rule, plus a check that the cited rule's condition actually holds on the record (fabricated grounds fail even when the verdict is right).
- **The transfer result** (§5.1): leave-case-out over all 34 policies, 72.9% exact [64.7, 80.9] vs 15.0% raw under identical bf16 greedy serving; ICL ladder 15.0 $\rightarrow$ 50.0 $\rightarrow$ 72.9.
- **The counterfactual control** (§5.2): 88.0% on flipped policies where priors score zero by construction — the model learned to *read the policy*, not the world's defaults.
- **The negative control and placement economics** (§5.3–5.4): the same recipe on byte-exact arithmetic derivation fails (0/6), survives a chain-of-thought-target ablation (still 0/6), and is matched by a *frontier* model failing the same gate on the same division fields without tools. This is a control, not a discovery — everyone routes arithmetic to tools — but it justifies going one step further than per-run tool calls: compiling the step to code, which removes the LLM from repeat runs entirely and yields a measured second-run break-even.

2. Related Work

To be expanded; verified positioning from our survey:

Policy internalization vs. policy-in-context. Multimodal Policy Internalization (TriMPI, 2510.09474) trains policies *into* model parameters so the policy need not be in context at inference — the exact opposite maintenance bet from ours. Its "Policy Override" setting (§5.2 there) supplies an updated policy in context at inference as a *generalization probe* of the internalized model; we make in-context application the training objective itself. Policy Reasoning Traces (2509.23291) improves compliance assessment on specific policies such as HIPAA and GDPR with per-policy instance-level train/test splits; cross-policy transfer appears only as an auxiliary analysis (their §4.2/App. G), not as the held-out-policy protocol we use. GuideBench (2505.11368) benchmarks robustness to rule updates for LLM agents.

Policy-following benchmarks. τ -bench (2406.12045) evaluates state-of-the-art agents given domain-specific policy guidelines; RuleArena's (2412.08972) findings distinguish failures of rule identification from failures of the computations the rules require — directly anticipating our §5.3 boundary; CRMarena-Pro (2505.18878) assesses LLM agents across enterprise scenarios, including confidentiality awareness under prompted policies. A contemporaneous enterprise benchmark ("Beyond the All-in-One Agent", 2605.08761, *abstract verified*) independently builds rule-engine-judged enterprise decisions; it is evaluation-only and has no discretion bands. DMN (OMG standard) has long standardized ordered decision rules with FIRST-hit semantics; our DSL is deliberately in that family — the ML contribution is the transfer measurement, not the rule language.

SFT and arithmetic. Faith and Fate (2305.18654) shows Transformers fail at compositional multi-step arithmetic, reducing reasoning to linearized pattern matching; "SFT Memorizes, RL Generalizes" (2501.17161) shows SFT tends to memorize and struggles out-of-distribution; PAL (2211.10435) offloads the solution step to a Python interpreter, the ancestral form of routing computation to code. Our contribution here is not the negative result itself but its *pairing*: the same recipe, corpus scale, and gate that fail on arithmetic succeed on rule application — and a tool-less frontier model fails the same arithmetic gate, sharpening "small models can't compute" into "inference can't compute."

Small-model deployment. The SLM position paper of Belcak et al. (2506.02153) argues small models are the future of agentic AI and outlines an LLM-to-SLM agent conversion algorithm; we supply a trained-skill recipe and a transfer measurement for one high-value slot (policy application) and a deterministic gate that even frontier models fail for another (derivation), yielding a placement rule rather than a preference.

3. Task and Corpus

3.1 Decision cases

A *case* declares one recurring organizational decision: the deciding role, an **ontology** (entities, attributes, relations), a feature space, **ordered rules** (when condition $\rightarrow$ verdict, route, parameters, rationale), optional **discretion bands** (defer: `slm_recommend`, with the band's limits as parameters), and a fallback. Example (abridged):

```
id: sales-discount-approval          # org: 영업 · role: 영업대표
rules:
- {name: min_margin_violation, when: margin_pct - requested_discount_pct < 10, verdict: reject}
- {name: strategic_retention_band,
  when: segment == strategic and churn_risk == high and requested_discount_pct <= 12,
  verdict: approve, route: 본부장, set: {max_discount_pct: 12}}
```

```

- {name: over_director_threshold, when: requested_discount_pct > 10, verdict: escalate,
  route: 본부장}
- {name: standard_auto, when: requested_discount_pct <= 5, verdict: approve, route:
  auto}
- {name: gray_band_recommend, when: requested_discount_pct <= 10,
  defer: slm_recommend, route: 팀장, set: {recommend_between: [5, 10]}}
fallback: {verdict: escalate, route: 본부장}

```

The corpus holds **34 cases across 10 functions** (sales, finance, support, HR, procurement, legal, IT operations, marketing, logistics, security). A deterministic engine samples records from each case's feature space and applies first-match semantics to produce **100 labeled decisions per case** (3,400 total), each with the applied rule's name, its condition, and a rationale. Generation is seeded and byte-reproducible.

3.2 Policy-in-context format

The model receives, per instance: a system turn naming the organization, role and decision; the case's ontology and its **full ordered rule list** (YAML, verbatim); and the record. It must output a single JSON object: `{verdict, route, params, cited_rule, rationale}`. Nothing about any specific policy is in the weights; a rule edit changes behavior with no retraining.

3.3 Grading

An answer is **exact** iff verdict, route, and parameters equal the engine's decision *and* the cited rule is the first-matching rule *and* that rule's condition actually holds on the record (the last check fails fabricated grounds even when the verdict is guessed right). A **relaxed** variant accepts a cited rule that is not first-match but whose condition genuinely holds with all other fields correct — reported alongside, since first-match is a convention, not a semantic requirement. We also report verdict-only accuracy. Because held-out items cluster by policy, all confidence intervals are **case-cluster bootstrap** (10,000 resamples over cases).

3.4 Honest scope

The corpus is synthetic and self-graded: the engine that labels is the engine that grades, the policies are already machine-readable, and rule-condition field names match record keys — the entity-alignment difficulty of prose policies is absent, as are noisy or missing fields and annotator disagreement. We therefore claim transfer of *policy application on declarative policies*, a deliberately clean substrate; §6 discusses the path to prose policies. (These limitations were surfaced by an internal adversarial audit and we keep them in the paper rather than sanding them off.)

4. Experimental Setup

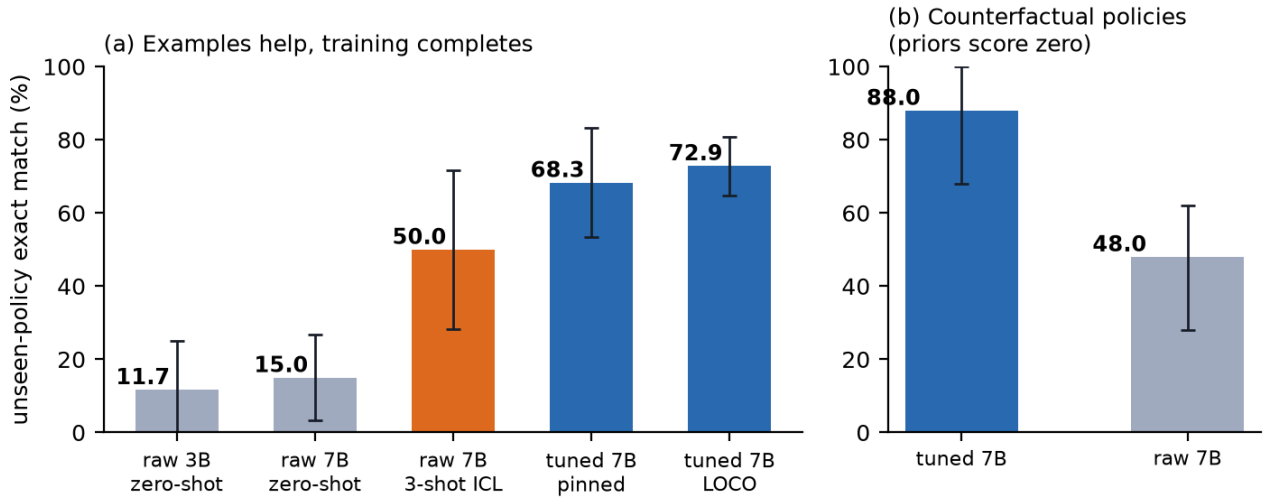
Model & training. Qwen2.5-7B-Instruct, QLoRA ($r=16$, $\alpha=32$, dropout 0.05, all linear projections), completion-only loss, 2 epochs, LR $1e-4$ cosine, bf16 compute on one RTX 4090 (≈ 12 min/run). Training data: 28 cases $\times$ 40 instances (1,120 rows) per split; 6 cases held out *entirely* (their policies never appear in any training row). Environment pinned (requirements-train.txt); every run writes a manifest (seed, epochs, data and adapter SHA-256, package versions).

Serving & fairness. All models — tuned *and* raw baselines — are evaluated through the same bf16 transformers server with greedy decoding. (An earlier protocol served raw baselines 4-bit quantized via a different stack; re-running them under identical serving moved raw 7B from 16.7% to 15.0%, i.e., the quantization confound does not explain the gap.)

Evaluation splits. (a) *Pinned split*: 6 held-out cases $\times$ 10 instances (60 items), plus 56 seen-policy/unseen-instance items. (b) *Leave-case-out (LOCO)*: 6 folds re-trained with the same recipe so that **every one of the 34 policies is unseen exactly once** (340 items, 34 clusters).

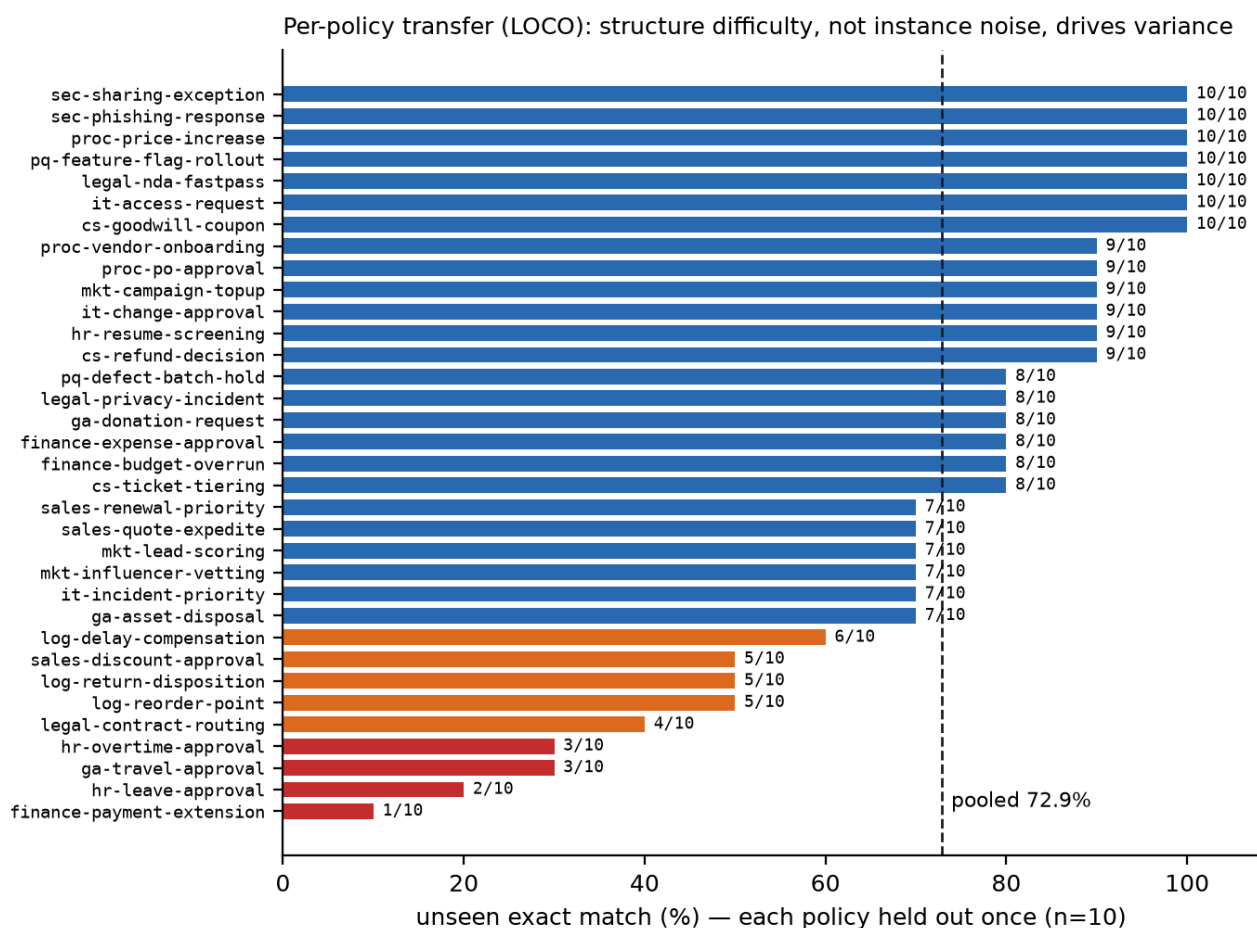
5. Results

5.1 Transfer to unseen policies



model (identical bf16 greedy serving)	unseen exact	95% CI (cluster)	verdict	relaxed
raw 3B, zero-shot	11.7%	[0.0, 25.0]	50.0%	11.7%
raw 7B, zero-shot	15.0%	[3.3, 26.7]	53.3%	15.0%
raw 7B, 3-shot ICL (worked examples from other cases)	50.0%	[28.3, 71.7]	75.0%	51.7%
tuned 7B (pinned split)	68.3%	[53.3, 83.3]	86.7%	71.7%
tuned 7B (LOCO, all 34 policies unseen once)	72.9%	[64.7, 80.9]	87.4%	73.8%

LOCO fold accuracies range 65.0–80.0%: the pinned-split result is not an artifact of one lucky partition. Across 3 training seeds on the pinned split, unseen exact is $62.8\% \pm 4.8\text{pp}$ (60.0, 60.0, 68.3) — run variance is real and reported, and no seed approaches the raw baseline. In-context examples help substantially (15 $\rightarrow$ 50%) but fine-tuning adds $\sim 20\text{pp}$ over ICL while also removing the per-request example tokens. Per-policy scores range 1/10–10/10 under LOCO (Figure 2); policy structure difficulty, not instance noise, drives the CI width — approval policies with date/limit arithmetic in their conditions (leave, overtime, payment extension) sit at the bottom, foreshadowing the computation boundary of §5.3.



5.2 What was learned: the counterfactual control

We flip approve↔reject in every rule (and the fallback) of the held-out cases and evaluate **only on records where the flipped policy decides differently from the original** — on this subset, answering from priors or from the original policy scores zero by construction.

model	counterfactual exact (n=50)
tuned 7B	88.0%
raw 7B	48.0%

The tuned model follows the flipped policy. Combined with §5.1, this is direct evidence that fine-tuning taught **application of the in-context policy**, not policy content and not world priors. (One case has no approve/reject verdicts to flip and is excluded; 88.0% exceeds the model's normal unseen accuracy because the discriminating records exercise explicit verdict rules, the easiest citation targets.)

5.3 Negative control: the same recipe cannot learn computation — and neither can prompting

We ran the identical recipe (same trainer, gate, corpus scale: 300 examples) on a *derivation* task from the same product family: regenerate a customer's renewal-pricing files (22 numeric fields: seat rounding, growth percentages, band lookups, discount caps, monthly and annual totals) byte-exactly for held-out customers, judged by a deterministic gate.

arm	gate (exact)	numeric-field accuracy	where it fails
SFT 7B (plain targets)	0/6	—	computed values wrong
SFT 7B (chain-of-thought targets)	0/6	86.4% (114/132)	two long-division fields; one band-cascade error
frontier model, same prompt, no tools	0/6	93.6% (103/110)	almost solely the two long-division fields

We stress that this is a **control, not a discovery**: that LLMs should route arithmetic to tools is settled practice. Its role here is threefold. (i) It closes the "train harder" objection against our tier map — the 0/6 is not an artifact of reasoning-free targets (CoT ablation) nor of model scale (the frontier fails the same gate on the same two long-division fields while getting every band and discount rule right; the recorded agent originally passed because it computed with tools). (ii) The field-level scores show what the exact gate hides: inference gets 20–21 of 22 fields; the failure is concentrated precisely where computation, not rule-reading, is required. (iii) It motivates going one step further than the standard remedy. A per-run **tool call keeps the LLM in the loop on every execution** — it re-decides which tool to invoke (tokens), can decide differently (nondeterminism), and re-orchestrates the sequence each time. Compiling the step to the **code tier makes that decision once**, at compile time; repeat runs then execute with no LLM present. The resulting placement rule: derivations → code tier; policy judgments → trained-SLM tier behind the deterministic gate; declared discretion bands → recommendation + human approver.

5.4 Placement economics: break-even at the second run

Lowering LLM steps out of the loop is a compiler bet — pay once, win per execution — so the honest question is where the break-even sits. Measured on the renewal pipeline (unique tokens, i.e., not double-counting the cumulative context an agent resends each turn): the recorded agent session costs **23,614 frontier tokens per run**; after compilation and SLM promotion a run consumes **0 frontier tokens** (4,208 tokens on a local model at \$0 marginal cost, 7.4× faster, outputs reproduced 7/7 under a deterministic benchmark). Because the compiler's input is a work session the organization had to run once anyway, and compilation itself is deterministic local computation, the frontier-token **break-even is the second run** — compared with the ≈17-transaction break-even reported for optimization-heavy compilation (Compiled AI, 2604.05150 — "breaking even with runtime inference at approximately 17 transactions"), where the compiler spends LLM tokens searching. Escalations for genuinely new parameter values each cost one frontier call, are cached with upstream-output fingerprints, and amortize to zero on repeats. The hidden denominator is the human cost of declaring the ontology and policy; the pipeline therefore pays off in proportion to a decision's repetition frequency, which is exactly the regime organizations automate first — and the record-first design (the spec is extracted from the paid-for session, not authored from scratch) is what keeps that denominator small.

6. Discussion & Limitations

Why not internalize? A weights-internalized policy answers fast but ages instantly and cannot explain a rule edit. Policy-in-context inverts the maintenance economics: the policy file is the deployment artifact; retraining is only needed when the *skill* (not the policy) drifts.

Beyond policy decisions. Nothing in the recipe is specific to approval workflows: the training rows are (document, record, grounded structured answer) triples, and the counterfactual control generalizes to any document class whose semantics can be programmatically perturbed. Runbooks (procedure-in-context, execution-skill-in-weights) and rubrics (criteria-in-context, grading-skill-in-weights) are the nearest neighbors; each needs

its own deterministic grader, which is the actual bottleneck for widening the claim. This paper is also one tier of a larger compiled-pipeline system — recorded agent sessions lowered to code/rule/cache tiers, with the trained-SLM tier of this paper occupying the judgment slot behind a promotion gate — whose end-to-end economics (§5.4) motivate but do not depend on the transfer result.

document class	content in context	skill in weights	deterministic grader needed	status
declarative decision policies	ontology + ordered rules	ordered evaluation, first-match, grounded citation	rules engine (exists)	measured — this paper
escalation matrices	routing table + thresholds	table lookup + boundary cases	table-lookup checker	near — several catalog cases are of this shape
compliance checklists	itemized requirements	item-by-record matching	per-item rule checker	not started
grading rubrics	criteria + level descriptors	criterion application, level assignment	answer-key grader	not started
support runbooks	procedure + preconditions	step execution & state tracking	state-machine checker	not started

Path to prose policies. Our substrate removes entity alignment (rule field names match record keys). The natural next experiment keeps the trained applier fixed and inserts a policy-compilation step (prose → declarative rules) in front — measuring how much of the 72.9% survives noisy compilation, and whether the counterfactual control still holds.

Circularity. Labels and grades come from the same engine. We mitigated with the grounds-hold check (fabricated citations fail), the relaxed metric (first-match convention separated from semantics), the counterfactual control (priors zeroed), and full pinning — but human-annotated decisions with inter-annotator agreement remain future work.

Scale. 34 policies, one 7B base model, one language pair (Korean prompts, English/Korean fields). Cluster CIs are honest about this: [64.7, 80.9] is the claim, not 72.9%.

7. Reproducibility

Corpus generator, trainer, evaluation, all raw results and this draft live in one repository: [examples/org/](#) (catalog, engine, corpus), [tools/remote-train/](#) (pinned trainer + manifests, LOCO and control runners), [examples/org/decision-slm/](#) (RESULTS.md, eval_history.json, loco/results.json, c3c_controls.json), [examples/demo/customer-renewal-bench/slm-training/](#) (arithmetic arms, controls.json). Every table in this paper is regenerable from a seeded generator and stored model outputs.

TODO before submission

- ☐ Related-work prose (§2 is currently a positioning skeleton).
- ☐ Decide whether naming OpenWorkLang/OpenWorkCompiler in §1 must be anonymized for review.
- ☒ refs.bib authors completed from arXiv pages (four highly-cited entries kept as first-author + others).
- ☐ Decide the venue and convert to its LaTeX template (ACL style files).
- ☐ Limitations section as a separate ACL-required section (lift from §3.4/§6).

- ☐ Consider adding the 14B/other-base-model column if GPU time allows.